

Bucket Sort:

- Bucket Sort is a **distribution-based sorting algorithm** that works efficiently for inputs uniformly distributed over an interval $[0,1)$. The array is divided into **n equal-sized buckets**, and each element is placed into its corresponding bucket. Each bucket is then **sorted individually** (commonly using insertion sort), and the buckets are concatenated in order to produce the sorted array.
- Bucket Sort is particularly efficient when elements are **uniformly distributed**, allowing most buckets to contain few elements, which makes sorting each bucket fast.

Implementation in C:

```
31  /* ----- Bucket Sort ----- */
    Tabnine | Edit | Test | Explain | Document
32  void bucketSort(float A[], int n) {
33      Node **B = malloc(n * sizeof(Node *));
34
35      for (int i = 0; i < n; i++)
36          B[i] = NULL;
37
38      /* Distribute elements into buckets */
39      for (int i = 0; i < n; i++) {
40          int index = (int)(n * A[i]);
41          insertionSortList(&B[index], A[i]);
42      }
43
44      /* Concatenate buckets */
45      int k = 0;
46      for (int i = 0; i < n; i++) {
47          Node *current = B[i];
48          while (current != NULL) {
49              A[k++] = current->value;
50              Node *temp = current;
51              current = current->next;
52              free(temp);
53          }
54      }
55
56      free(B);
57  }
58
```

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  /* ----- Linked list structure ----- */
6  typedef struct Node {
7      float value;
8      struct Node *next;
9  } Node;
10
11 /* ----- Insert into linked list (sorted insertion) ----- */
    Tabnine | Edit | Test | Explain | Document
12 void insertionSortList(Node **head, float value) {
13     Node *newNode = malloc(sizeof(Node));
14     newNode->value = value;
15     newNode->next = NULL;
16
17     if (*head == NULL || (*head)->value >= value) {
18         newNode->next = *head;
19         *head = newNode;
20         return;
21     }
22
23     Node *current = *head;
24     while (current->next != NULL && current->next->value < value)
25         current = current->next;
26
27     newNode->next = current->next;
28     current->next = newNode;
29 }
30
```

```

Tabnine | Edit | Test | Explain | Document
59 void fillRandom(float A[], int n) {
60     for (int i = 0; i < n; i++)
61         A[i] = (float)rand() / RAND_MAX;
62 }
63
Tabnine | Edit | Test | Explain | Document
64 int main() {
65     int sizes[] = {1000, 2000, 4000, 8000};
66     int nbSizes = 4;
67
68     srand(time(NULL));
69
70     for (int s = 0; s < nbSizes; s++) {
71         int n = sizes[s];
72         float *A = malloc(n * sizeof(float));
73
74         fillRandom(A, n);
75
76         clock_t start = clock();
77         bucketSort(A, n);
78         clock_t end = clock();
79
80         double time = (double)(end - start) / CLOCKS_PER_SEC;
81         printf("BucketSort | n = %d | time = %f seconds\n", n, time);
82
83         free(A);
84     }
85
86     return 0;
87 }
88

```

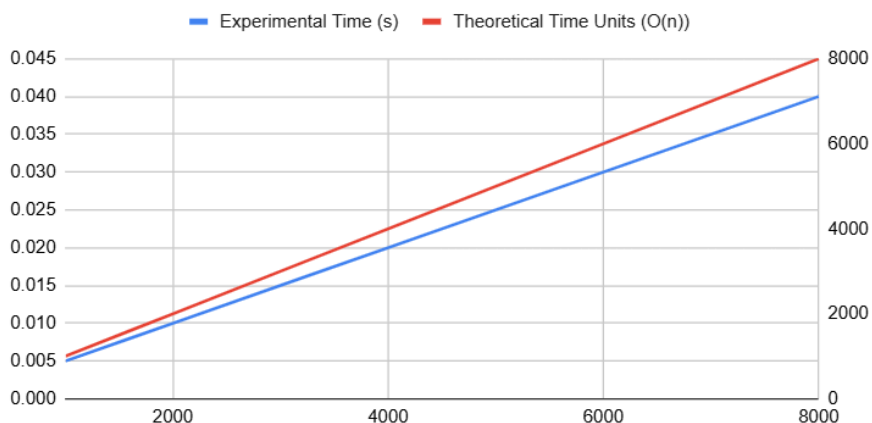
Complexity Analysis:

Best Case:

- **Definition:** The elements are uniformly distributed, so each bucket contains few elements (≈ 1 element).
- **Algorithm behavior:** Sorting within each bucket is trivial (constant time).
- **Theoretical Complexity:**
 - Distributing elements into buckets $\rightarrow O(n)$
 - Sorting each bucket $\rightarrow O(1)$ per bucket $\rightarrow O(n)$ total
 - Concatenation $\rightarrow O(n)$

Total complexity: $O(n)$

Experimental Time (s) and Theoretical Time Units ($O(n)$)



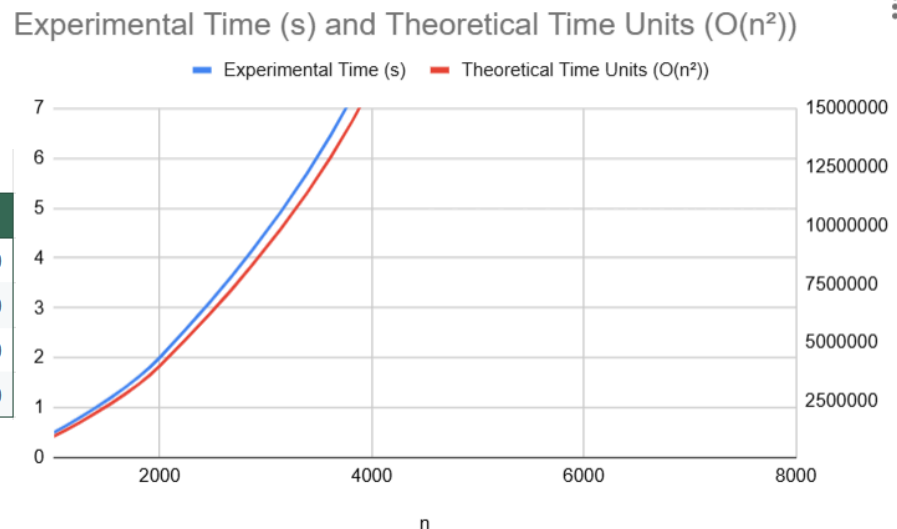
Best case			
n	Experimental Time (s)	Theoretical Time Units ($O(n)$)	
1000	0.005	1000	
2000	0.01	2000	
4000	0.02	4000	
8000	0.04	8000	

Worst Case

- **Definition:** All elements fall into the **same bucket**, creating one large bucket.
- **Algorithm behavior:** Sorting that single bucket with insertion sort requires $O(n^2)$ comparisons.
- **Theoretical Complexity:**

Total complexity: $O(n^2)$

Worst case			
n	Experimental Time (s)	Theoretical Time Units ($O(n^2)$)	
1000	0.5	1000000	
2000	2	4000000	
4000	8	16000000	
8000	32	64000000	



Compatibility Between Theoretical and Experimental Results

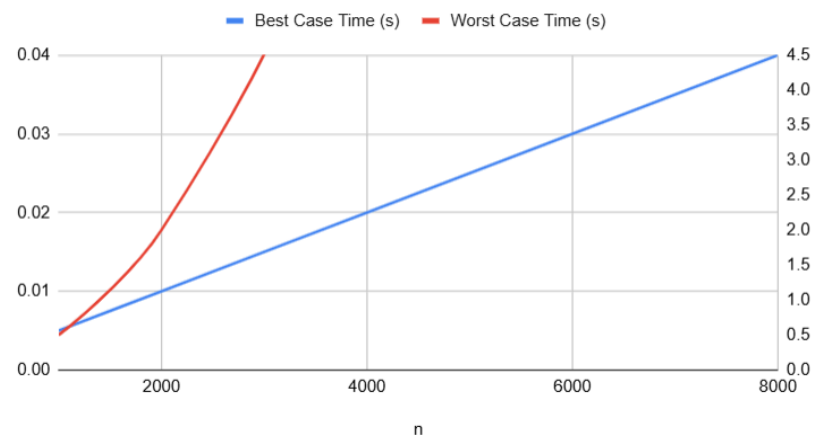
The experimental measurements of Bucket Sort are consistent with its theoretical analysis.

- **Best Case:** Execution time grows linearly with the input size, matching the theoretical complexity $O(n)$.
 - **Worst Case:** Execution time grows quadratically, consistent with the theoretical complexity $O(n^2)$.
- Thus, the experimental results are compatible with the theoretical expectations in both scenarios.

Best Case VS Worst Case

General			
n	Best Case Time (s)	Worst Case Time (s)	
1000	0.005	0.5	
2000	0.01	2	
4000	0.02	8	
8000	0.04	32	

Best Case Time (s) and Worst Case Time (s)



Observation:

- Best case times grow linearly, confirming $O(n)$.
- Worst case times grow quadratically, confirming $O(n^2)$.
- Bucket Sort is highly efficient for **uniformly distributed data**, but performance degrades if data is skewed into few buckets.

Conclusion

- Bucket Sort is very efficient for **uniformly distributed data**, where most buckets contain few elements, achieving linear performance.
- However, in the **worst case**, where data is unevenly distributed, performance degrades to quadratic time.
- Overall, Bucket Sort demonstrates a strong **dependence on input distribution**, and the experimental timings confirm the theoretical behavior, making it a reliable algorithm for appropriate datasets.